Campus Management System

Implementing Data Structures and Algorithms to Solve Real-World Problems

Data Structures & Algorithms

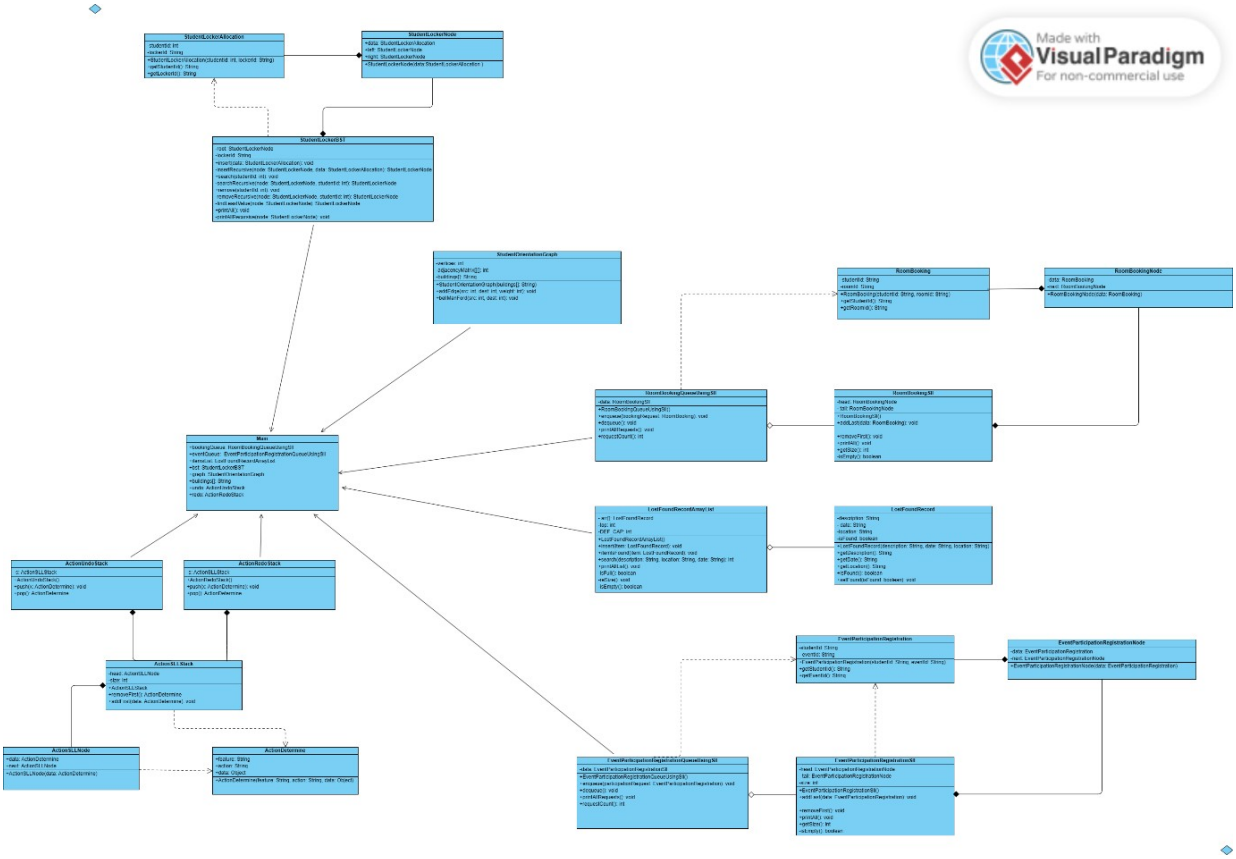
Spring 2025

Contents

1. Class Diagrams and User Operations....3
2. ADT Selection and Justification....6
3. Time Complexity Analysis....9
4. Graph Representation and Bellman-Ford Algorithm....10
5. Information Hiding, Encapsulation, and ADTs as a Basis for OOP....13
6. Comparison with Alternative Data Structures....14
References....16

1. Class Diagrams and User Operations

Class diagram of the whole system



Feature 1: Track Lost/Found Records

- **Used ADT:** ArrayList

Class diagram



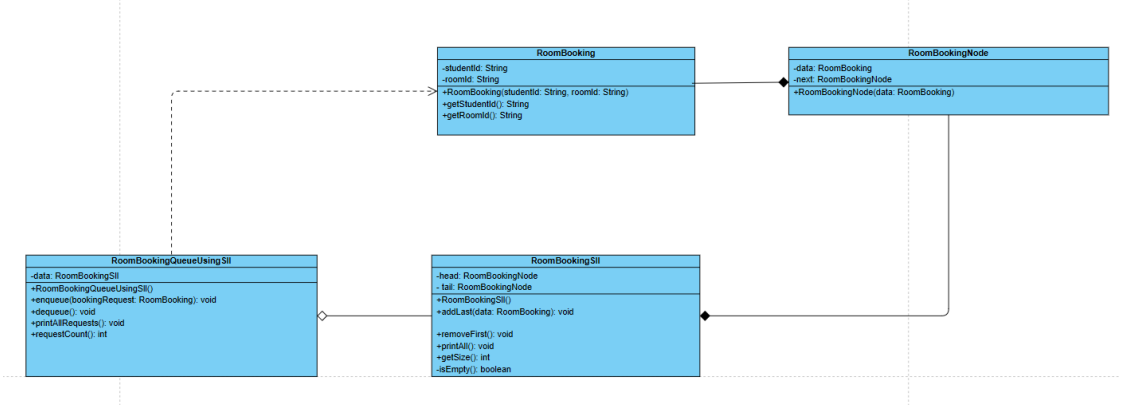
Valid operations for users

1. A student can track lost or found items using search(String description, String location, String date), O(n).
2. Staff can add lost items using insert(LostFoundRecord item), O(1).
3. Staff can set items as found using itemIsFound(LostFoundRecord item), O(n).
4. Staff can view all lost or found items using printAllList(), O(n).

Feature 2: Room Booking

- **Used ADT:** Queue using LinkedList with tail

Class diagram



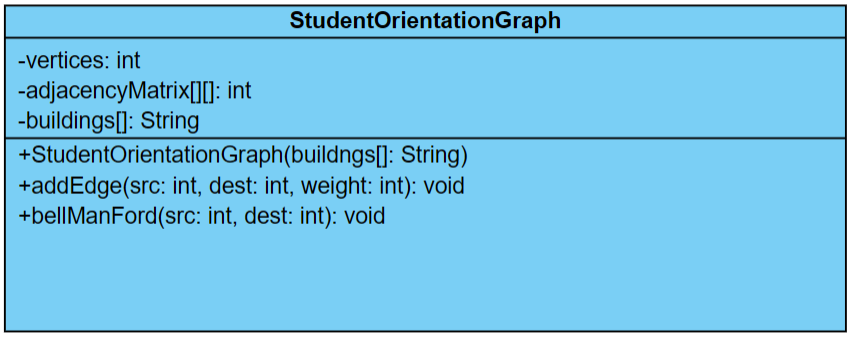
Valid operations for users

1. A student can add a room booking request using enqueue(RoomBooking bookingRequest), O(1).
2. Staff can approve a room booking request using dequeue(), O(1).
3. Staff can print all room booking requests using printAllRequests(), O(n).
4. Staff can view how many requests were received using requestCount(), O(1).

Feature 3: Student Orientation Help

- **Used ADT:** Graph using the Bellman-Ford algorithm

Class diagram



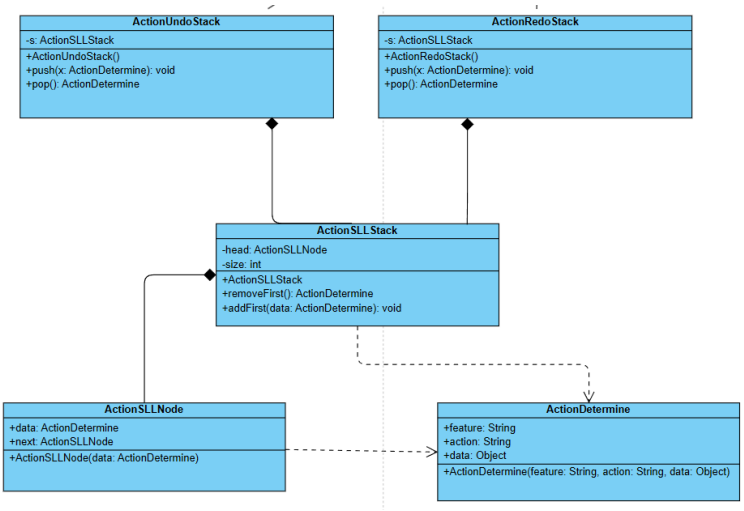
Valid operations for users

1. Students can find the shortest path between buildings using bellManFord(int src, int dest), $O(n^3)$.

Feature 4: Undo/Redo

- **Used ADT:** Two stacks using LinkedList

Class diagram



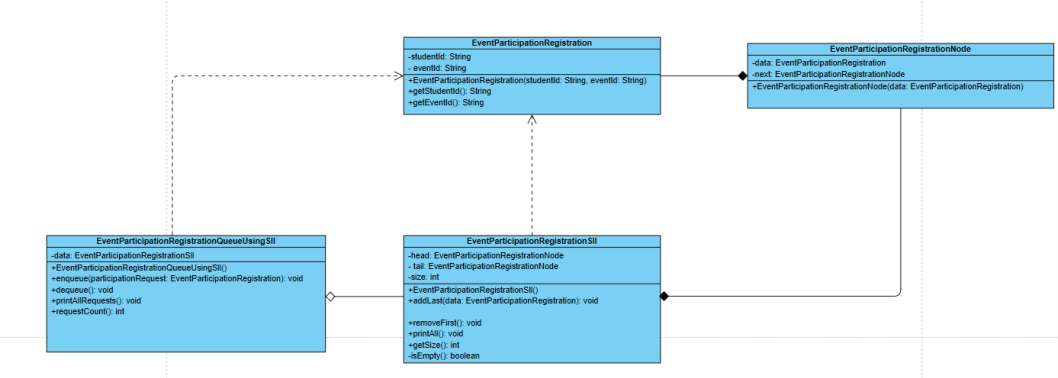
Valid operations for users

1. Any user action (adding a room booking request, adding an event registration request, or assigning a locker) can be undone using undo.pop(), O(1).
2. Any undone action can be redone using redo.pop(), O(1).

Feature 5: Event Participation Registration

- **Used ADT:** Queue using LinkedList with tail

Class diagram



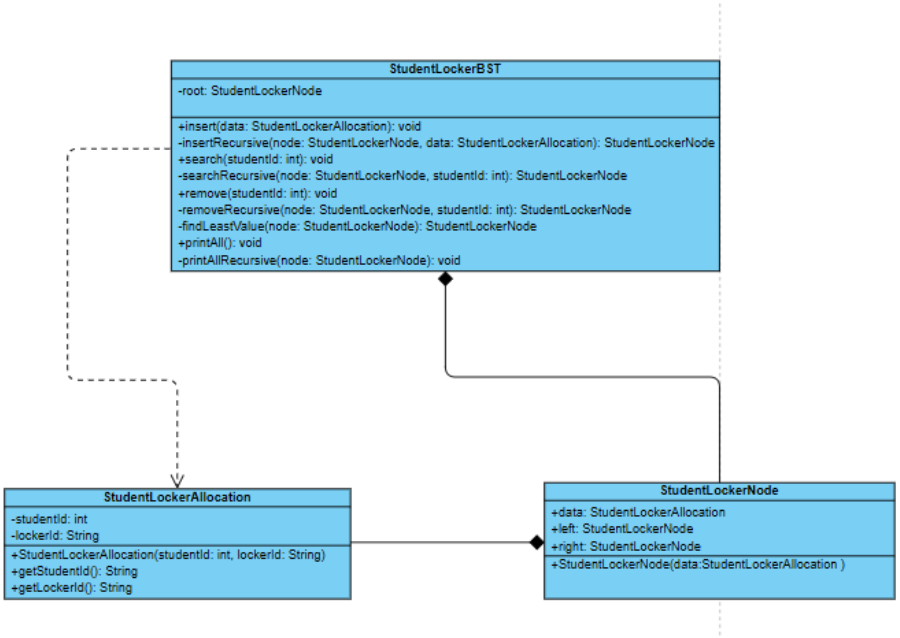
Valid operations for users

1. A student can send an event participation registration request using enqueue(EventParticipationRegistration participationRequest), O(1).
2. Staff can approve a student's request using dequeue(), O(1).
3. Staff can print all received requests using printAllRequests(), O(n).
4. Staff can view how many requests were received using requestCount(), O(1).

Feature 6: Student Locker Allocation

- **Used ADT:** Tree (binary search tree)

Class diagram



Valid operations for users

1. A student can search for their locker using search(int studentId), O(log n).
2. Staff can assign a locker to a student using insert(StudentLockerAllocation data), O(log n).
3. Staff can remove a locker from a student using remove(int studentId), O(log n).
4. Staff can view all reserved lockers using printAll(), O(n).

2. ADT Selection and Justification

Feature 1: Track Lost/Found Records

For feature one, I needed an efficient way to let staff add lost items, set these items as found, and view all lost or found records, and to let students track their lost items. I used an ArrayList because it gives easy access to the items in the list, which makes it easier for students to track their items and for staff to update items, add items, and view all records. An ArrayList was the best choice because it grows dynamically, so we do not have to worry about space, and the system can hold any number of items. It is also considered space-efficient when working with large amounts of data.

Use cases

- A student loses their AirPods and goes to the staff to have the lost item added to the list.
- The staff add the lost item by providing a description of the item, the location where it was lost, and the date when it was lost.
- When the item is found, the staff set it as found.
- The student can track their lost item by providing the description, location, and date. The student then sees the item with its status (found or still lost).
- Staff can view all lost or found items.

How this ADT meets user needs: Students do not need to track all lost or found items, only their own, so I let them track an item by providing only its description, location, and date. Staff needed to add lost items to the system easily, so they only enter the description, location, and date of the item. Staff also needed to set found items as found so that students can easily check the status of their item, and sometimes staff need to see all items, which I also provided by entering the number of the sub-feature in the menu.

Feature 2: Room Booking

For feature two, students needed to request bookings of private study rooms, and the first student who requests a room should be the first whose request is approved by staff. Staff also needed to view how many room booking requests they may accept, based on the number of rooms available, and to view all booking requests. So I needed something with FIFO (first in, first out) behaviour, and I used a queue. The queue was built over a singly linked list with a tail, so that all operations take the least time: all operations are O(1) except printing, which is O(n). Staff can also easily see which request to approve first, because the next request to approve is always at the front.

Use cases

- A student sends a room booking request by providing their studentId and the roomId they need, and the request is added to the queue with enqueue.
- A staff member can then view all the requests using printAllRequests, so they see the first request, and they can also view the number of requests.
- A staff member can then approve the request of the first student who requested a room, which supports the FIFO behaviour. This is done using dequeue, and the request is removed from the queue when it is approved.

How this ADT meets user needs: The requests made by students are treated fairly, because the first student who requested a room is the first to be approved by staff. Staff also need to view the requests and the request count to make sure they only approve requests according to the rooms available.

Feature 3: Student Orientation Help

For feature three, students needed to find the shortest paths between campus buildings by entering the source and destination buildings, which is especially useful for guiding new students. I used a graph ADT, where each building is a vertex and the paths between buildings are edges. The edges are weighted to represent the distance in meters between buildings. I used the Bellman-Ford algorithm to find the shortest paths between the buildings, because it is suitable for weighted graphs and it is efficient.

Use cases

- A new student who does not know the buildings or where they are wants to find the shortest path between two buildings. They use feature 3 by providing the source building and the destination building, and the system shows the shortest path between these two buildings.

How this ADT meets user needs: Students need to find the shortest path accurately. The graph shows the paths between the buildings and lets us represent the distances between them, and by using Bellman-Ford, I made sure that students always get the shortest path between two buildings.

Feature 4: Undo/Redo

For feature four, users need to undo and redo some actions in the system, including adding a room booking request, adding an event registration request, and assigning a locker. This is helpful when a user makes a mistake and does something they did not want to do. I used two stacks, one for undo and one for redo, and each stack was built over a singly linked list. We needed LIFO (last in, first out) behaviour, because we undo the last action and redo the last undone action, so a stack is a perfect fit.

Use cases

- A student requests a room booking by mistake and wants to undo the request. They select undo, and the system removes it from the booking requests.
- The same student realises that they needed that room, so they redo what they undid, and the request is inserted back into the booking requests.

How this ADT meets user needs: Students and staff, who are the users of the system, sometimes perform an action by mistake, and the undo feature lets them remove that action from the system. Users may also change their mind about undoing an action, in which case they simply redo it and it is back in the system. All undo and redo operations take O(1), which makes the system more efficient and faster.

Feature 5: Event Participation Registration

For feature five, students needed to send event participation registration requests, and the first student who requests to participate should be the first whose request is approved by staff. Staff also needed to view how many event participation requests they may accept, based on the number of seats available for that event, and to view all event participation requests. So I needed something with FIFO behaviour, and I used a queue built over a singly linked list with a tail, so that all operations take the least time: all operations are O(1) except printing, which is O(n). Staff can also easily see which request to approve first, because the next request to approve is always at the front.

Use cases

- A student sends an event participation registration request by providing their studentId and the eventId they want to participate in, and the request is added to the queue with enqueue.
- A staff member can then view all the requests using printAllRequests, so they see the first request, and they can also view the number of requests.
- A staff member can then approve the request of the first student who requested to participate, which supports the FIFO behaviour. This is done using dequeue, and the request is removed from the queue when it is approved.

How this ADT meets user needs: The requests made by students are treated fairly, because the first student who requested to participate is the first to be approved by staff. Staff also need to view the requests and the request count to make sure they only approve requests according to the seats available.

Feature 6: Student Locker Allocation

For feature six, staff needed to allocate lockers easily and to support quick lookup, removal, and registration of lockers. Students also needed to check whether they have a locker assigned. I used a binary search tree to solve this problem, because it supports quick lookup, lets us store locker assignments, and lets us remove locker assignments easily. All operations are efficient and take O(log n), except printing, which takes O(n).

Use cases

- A staff member assigns a locker to a student using the insert method and providing the studentId and the lockerId.
- A staff member can then view all reserved lockers to avoid reserving the same locker for another student.
- When a student graduates or withdraws, a staff member can remove the locker assignment for that student by calling the remove() method with the student ID.
- A student can check whether they have a locker assigned, and if so, what the locker ID is, by calling the search() method with their studentId.

How this ADT meets user needs: Students needed quick lookup, and the tree provides it based on the student ID. Staff can also add, remove, and view locker assignments effectively and in an organised way, which the BST implementation provides.

Benefits and Weaknesses of the Selected ADTs

I used a queue over a linked list with a tail for features 2 and 5, and two stacks over a linked list for feature 4. This implementation has benefits, strengths, and weaknesses, which I explain in general and then relate to my code.

Queue over a linked list with a tail

There are many benefits to using a queue over a linked list with a tail. I developed the linked list code first, and it is reusable in many ADTs. When I built the queue, I only needed to know how it should behave (FIFO). Using a linked list with a tail ensures that the enqueue and dequeue operations take O(1) time, which is very efficient. The queue is also scalable, as it grows as needed, and it does not need to sort the requests because of its FIFO behaviour. In my code, thanks to the FIFO behaviour, students are treated fairly, because the first student who makes a request has priority to be approved first. Staff also do not have to order the requests in the queue, because they are already ordered from the oldest request to the newest.

There are also weaknesses. There is no direct access to the elements in the list, unlike in an ArrayList, although removal in an ArrayList would take O(n), which is worse than in our case. In my code, this means that staff cannot approve a request that is in the middle of the list; they must approve the requests in order.

Stack over a linked list

There are many benefits to using a stack over a linked list. Stack operations are efficient, as they take O(1) time when the stack is built over a linked list, so the feature is always fast and system performance stays high. A stack works as LIFO, which matches how undo and redo work: the last action performed is the first action to be undone. This makes the system easy to maintain and understand. The stack is also scalable, as it grows as needed, so we do not need to worry about memory size.

There are also weaknesses. There is no direct access to the elements in the list, unlike in an ArrayList. In my code, this means that users cannot undo or redo an action that is in the middle of the list; they must undo or redo the actions in order.

3. Time Complexity Analysis

Feature 1: ArrayList

- **insert():** This method takes O(1) time, because a value is inserted at a specific index. When resizing is needed, it takes O(n) to perform the resizing.
- **itemIsFound():** This method takes O(n) time, because to access the element needed, we must go through the list and compare the input with the actual data in the list. When a match is found, its status is set to true. In the best case, it takes O(1), if the item is found at the beginning of the array.
- **search():** This method takes O(n) time, because to access the element needed, we must go through the list and compare the input with the actual data in the list. In the best case, it takes O(1), if the item is found at the beginning of the array.
- **printAllList():** This method takes O(n) time, because to print all elements in the list, we must go through all of them and print them one by one.
- **isFull():** This method takes O(1), because it only checks whether the array is full. It is used in the resize method.
- **isEmpty():** Like isFull(), this method takes O(1), because it only checks whether the list is empty.

- **reSize():** This method takes O(n) time, because we need to copy the elements in the list to the new list, and then we perform the resizing.

Feature 2: Queue over LinkedList with Tail

- **enqueue():** This method takes O(1) time, as it uses the insert() method of the linked list, and because the linked list has a tail, we only need to update pointers to add new data at the tail.
- **dequeue():** This method takes O(1) time, because the removal is done from the head, which is known, so we only update the pointers.
- **printAllRequests():** This method takes O(n) time, because to print all elements in the list, we must go through all of them and print them one by one.
- **requestCount():** This method takes O(1) time, because it only returns a value.

Feature 3: Graph with Bellman-Ford

- **addEdge():** This method takes O(1) time, because it simply sets a value at a specific location in the adjacency matrix.
- **bellManFord():** This method takes $O(n^3)$, because it has three nested loops and each one runs n times, so $n \times n \times n$ is n^3.

Feature 4: Stack over LinkedList

- **push():** This method takes O(1) time, because it simply adds a new node at the head.
- **pop():** This method takes O(1) time, because it simply removes a node from the head.

Feature 5: Queue over LinkedList with Tail

Feature 5 uses the same ADT as Feature 2, so the complexity of its operations is the same, for the same reasons: enqueue() takes O(1), dequeue() takes O(1), printAllRequests() takes O(n), and requestCount() takes O(1).

Feature 6: BST

- **insert():** This method takes O(log n), because it goes down the tree, cutting the tree in half each time.
- **remove():** This method takes O(log n), because it goes down the tree, cutting the tree in half each time.
- **search():** This method takes O(log n), because it goes down the tree, cutting the tree in half each time.
- **printAll():** This method takes O(n) time, because to print all elements, we must go through all of them and print them one by one.

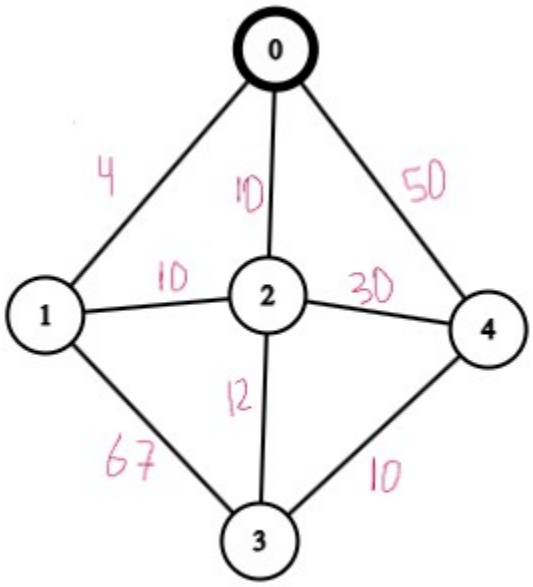
4. Graph Representation and Bellman-Ford Algorithm

In my graph, I used strings to define the building names, and numbers represent the buildings.

```
String buildings[] = {"Red Building", "Blue building", "W building", "Iman Lab", "Workshops"};
StudentOrientationGraph graph = new StudentOrientationGraph(buildings);
graph.addEdge(0, 1, 4);
graph.addEdge(0, 2, 10);
graph.addEdge(0, 4, 50);
graph.addEdge(1, 2, 10);
graph.addEdge(1, 3, 67);
graph.addEdge(2, 3, 12);
graph.addEdge(2, 4, 30);
graph.addEdge(3, 4, 10);
```

- 0 – Red Building
- 1 – Blue Building
- 2 – W building
- 3 – Iman Lab
- 4 – Workshops

Graph representation



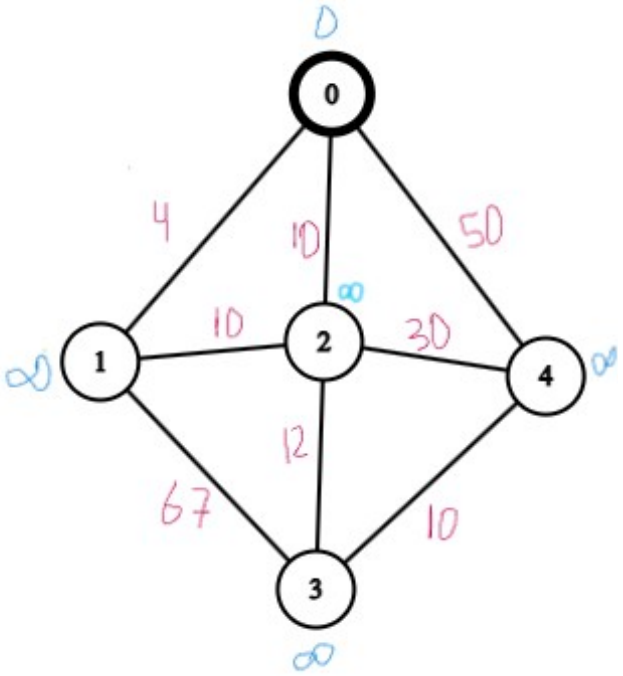
The implemented algorithm is Bellman-Ford with an adjacency matrix.

Adjacency matrix

	0	**1**	**2**	**3**	**4**
0	0	4	10	0	50
1	4	0	10	67	0
2	10	10	0	12	30
3	0	67	12	0	10
4	50	0	30	10	0

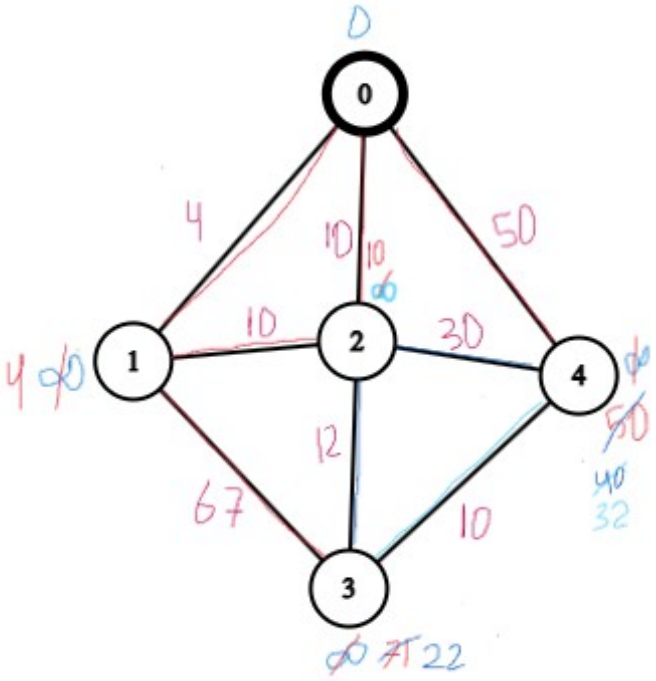
The number of iterations needed to relax all edges is V − 1, which is 5 − 1 = 4.

Before everything, we prepare a list of all edges, set the distance to every node to infinity, and set the starting node to 0. Then we relax all edges 4 times.

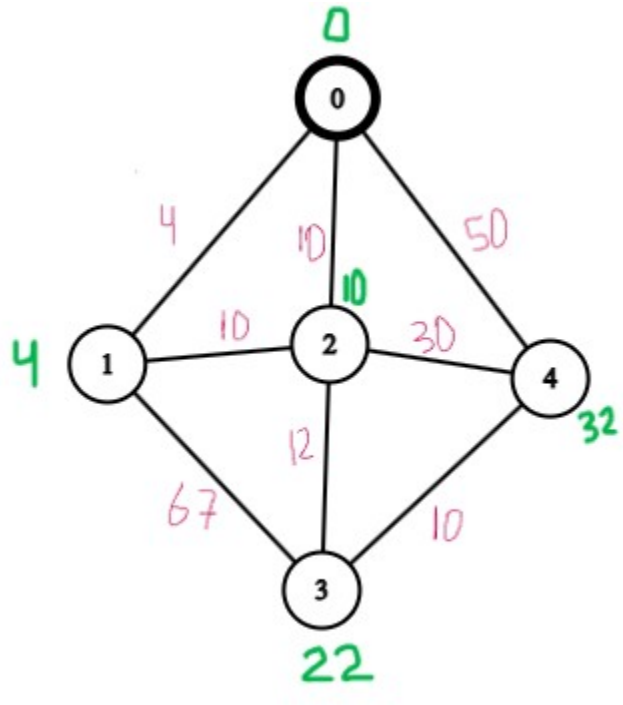


Edges: (0,1), (0,2), (0,4), (1,2), (1,3), (2,3), (2,4), (3,4)

First iteration



Second, third, and fourth iterations



The distances remain the same as in the first iteration in the second, third, and fourth iterations.

5. Information Hiding, Encapsulation, and ADTs as a Basis for OOP

Information hiding and encapsulation were very important in building a system like the campus management system. These principles helped me in many ways and made my code easier to organise and understand.

Information Hiding

Information hiding means that the details of an ADT are hidden. For example, the RoomBookingSll ADT hides some of its internal fields (head, tail, and size) by making them private, so users cannot interact with these fields directly, but they can interact with the methods that use them. By doing this, I made sure that all changes to the data go through the methods, which makes the system more efficient and reliable. For example, no one can set the head of RoomBookingSll to null, because if it were set to null, the whole list would be lost.

Encapsulation

Encapsulation is a way to restrict direct access to some components of an object. For example, the LostFoundRecord class has the attributes description, date, location, and isFound as private, so users cannot interact with these attributes directly. The class has getters because the LostFoundRecordArrayList class must work with these attributes, so users can only interact with the methods inside LostFoundRecordArrayList. This keeps the data consistent and keeps changes to these attributes controlled.

ADTs as a Basis for OOP

An ADT is seen as a basis for OOP because of the high similarity between them in some respects. An ADT is a data type whose internal representation is hidden from the client, which is also a foundation of OOP. OOP allows us to encapsulate the components of a class, which hides the

details from the client, just like the ADT definition of hiding the internal data representation and keeping only the necessary operations public.

Both ADTs and OOP also provide code reusability, as the methods of an ADT can be used in multiple parts of the main program without exposing the internal logic. An ADT also defines the operations that can be performed without exposing how they are implemented internally, just like abstraction in OOP. In addition, ADTs and OOP are not tied to a specific programming language, as a programmer can use them in many languages to solve problems.

However, ADTs are not a complete basis for OOP. Some view them as a basis because some OOP principles, such as encapsulation and abstraction, come from them, and an ADT can be seen as a conceptual foundation of OOP, but OOP has more principles, such as inheritance.

6. Comparison with Alternative Data Structures

For each feature, I compare the implementation I used with alternative data structures.

Feature 1: ArrayList

	ArrayList	LinkedList	LinkedList with tail	Doubly LinkedList	Doubly LinkedList with tail
insert()	O(1), and O(n) when resize is needed.	O(1) if addFirst, and O(n) if addLast.	O(1) always, because head is known and tail is known.	O(1) if addFirst, and O(n) if addLast.	O(1) in all cases, because head and tail are known.
itemIsFound()	O(n). The best case is O(1), when the item is found at the first position.	O(n). The best case is O(1), when the item is found at the first position.	O(n), we must go through the list in all cases. The best case is O(1), when the item is found at the first position.	O(n), we must go through the list in all cases. The best case is O(1), when the item is found at the first position.	O(n), we must go through the list in all cases. The best case is O(1), when the item is found at the first position.
search()	O(n). The best case is O(1), when the item is found at the first position.	O(n). The best case is O(1), when the item is found at the first position.	O(n), we must go through the list in all cases. The best case is O(1), when the item is found at the first position.	O(n), we must go through the list in all cases. The best case is O(1), when the item is found at the first position.	O(n), we must go through the list in all cases. The best case is O(1), when the item is found at the first position.
printAllList()	O(n), we must go through the list in all cases.	O(n), we must go through the list in all cases.	O(n), we must go through the list in all cases.	O(n), we must go through the list in all cases.	O(n), we must go through the list in all cases.
isFull()	O(1)	—	—	—	—
reSize()	O(n)	—	—	—	—
isEmpty()	O(1)	O(1)	O(1)	O(1)	O(1)
Space complexity	O(n), additional memory is required through resizing.	O(1), because in each insert, we add a node and a pointer.	O(1), because in each insert, we add a node and a pointer.	O(1), because in each insert, we add a node and a pointer.	O(1), because in each insert, we add a node and a pointer.

These alternatives do not differ much from the ArrayList.

Feature 2: Queue over LinkedList with Tail

	Queue over LinkedList with tail	Queue over LinkedList	Queue over doubly LinkedList	Queue over doubly LinkedList with tail	Queue over ArrayList
enqueue()	O(1), because we add last and tail is known.	O(n), because we add last and tail is not known.	O(n), because we add last and tail is not known.	O(1), because we add last and tail is known.	O(1), and O(n) when resize is needed.
dequeue()	O(1), because we remove first and head is known.	O(1), because we remove first and head is known.	O(1), because we remove first and head is known.	O(1), because we remove first and head is known.	O(1), because we remove first and head is known.
printAllRequests()	O(n), because we must go through all elements.	O(n), because we must go through all elements.	O(n), because we must go through all elements.	O(n), because we must go through all elements.	O(n), because we must go through all elements.

	Queue over LinkedList with tail	Queue over LinkedList	Queue over doubly LinkedList	Queue over doubly LinkedList with tail	Queue over ArrayList
requestCount()	O(1)	O(1)	O(1)	O(1)	O(1)
Space complexity	O(1), because in each enqueue, we add a node and a pointer.	O(1)	O(1)	O(1)	O(n), additional memory is required through resizing.

Feature 3: Graph with Bellman-Ford

	Graph with Bellman-Ford	Graph with Dijkstra
addEdge()	O(1), because we simply add it to a specific index in the adjacency matrix.	O(1), because we simply add it to a specific index in the adjacency matrix.
bellManFord()	$O(n^3)$, because there are three nested loops that go through everything.	$O(n^2)$, to check all possible neighbours.
Space complexity	$O(n^2)$, the adjacency matrix.	$O(n^2)$, the adjacency matrix.

Feature 4: Stack over LinkedList

	Stack over LinkedList	Stack over LinkedList with tail	Stack over doubly LinkedList	Stack over doubly LinkedList with tail	Stack over ArrayList
push()	O(1), because it simply adds a new node at the head.	O(1), because it simply adds a new node at the head.	O(1), because it simply adds a new node at the head.	O(1), because it simply adds a new node at the head.	O(1), but O(n) when resize is needed.
pop()	O(1), because it simply removes a node from the head.	O(1), because it simply removes a node from the head.	O(1), because it simply removes a node from the head.	O(1), because it simply removes a node from the head.	O(1), because we simply remove the last element.
Space complexity	O(1), because in each push, we add a node and a pointer.	O(1), because in each push, we add a node and a pointer.	O(1), because in each push, we add a node and a pointer.	O(1), because in each push, we add a node and a pointer.	O(n), additional memory is required through resizing.

Feature 5: Queue over LinkedList with Tail

Feature 5 uses the same ADT and the same operations as Feature 2, so the comparison is identical to the Feature 2 table above.

Feature 6: BST

	BST	Sorted ArrayList
insert()	O(log n), because it goes down the tree, cutting the tree in half each time.	O(n), because you need to find the correct position and then shift elements.
remove()	O(log n), because it goes down the tree, cutting the tree in half each time.	O(n), because you need to shift all the elements after the removal.
search()	O(log n), because it goes down the tree, cutting the tree in half each time.	O(log n), because binary search is used.
printAll()	O(n), because we must go through all elements.	O(n), because we must go through all elements.
Space complexity	O(1), because in each insert, we add a node with left and right pointers.	O(n), additional memory is required through resizing.

References

1. GeeksforGeeks (2022) *Implement Undo and Redo features of a Text Editor*. Available at: https://www.geeksforgeeks.org/implement-undo-and-redo-features-of-a-text-editor/
2. Studocu (n.d.) *Discuss the view that imperative ADTs are a basis for object*. Available at: https://www.studocu.com/row/document/advanced-programming-1651/solved-discuss-the-view-that-imperative-adts-are-a-basis-for-object
3. Studocu (n.d.) *Discuss the view that imperative ADTs are a basis for object - Computer Science (CS 1101)*. Available at: https://www.studocu.com/en-us/document/computer-science-cs-1101/solved-discuss-the-view-that-imperative-adts-are-a-basis-for-object
4. Yahya, H. (2024) *Data Structures and Algorithms Spring 25*. Available at: https://github.com/HikmatYahya/DataStructuresAndAlgorithmsSpring25
5. Louzi, M. (2022) HTU Slides.